

ADA Practica Final Integradora

Universidad EAFIT

Nombre de los estudiantes:

Juan Manuel Gallo López

David García Zapata

Nombre del docente:

Carlos Alberto Álvarez Henao

Asignatura:

Estructura de datos y algoritmos 2

Ciudad:

Medellín

Fecha:

07/05/2026

1. Introducción

1.1 Descripción del problema

Este proyecto aborda la optimización de infraestructura de telecomunicaciones mediante la aplicación de tres paradigmas fundamentales de algoritmos sobre datos reales. La problemática central consiste en tres subproblemas conectados:

- **Consultas eficientes:** Dado un conjunto de 7,043 solicitudes de servicio, ordenarlas y recuperar información específica mediante búsquedas parametrizadas.
- **Construcción de red óptima:** Diseñar una red de conectividad con costo mínimo entre 20 nodos de distribución.
- **Asignación de recursos:** Distribuir ancho de banda limitado (500 Mbps) entre 50 solicitudes activas maximizando el valor.

1.2 Contexto del Dataset

Se utiliza el dataset “Telecom Customer Churn” de Kaggle, que contiene información de 7,043 clientes de una empresa de telecomunicaciones con 21 atributos. Los atributos importantes que usamos en este trabajo son:

- **customerID:** Identificador único del cliente.
- **tenure:** Meses de antigüedad del cliente (Esta variable la usamos para hacer el ordenamiento)
- **MonthlyCharges:** Cargos mensuales
- **TotalCharges:** Cargos totales acumulados
- **Churn:** Indicador de abandono del servicio (Puede ser Yes o No)

1.3 Justificación de paradigmas

- **Módulo A:** En este módulo realizamos búsqueda parametrizada en datos ordenados. El paradigma que usamos es divide y vencerás. Justificación: El módulo consta de dos fases. Primero, usamos MergeSort (divide y vencerás) para ordenar los 7,043 registros por antigüedad descendente en $O(n \log n)$ de forma determinística, sin depender del orden inicial. Luego, ejecutamos búsqueda binaria recursiva (también divide y vencerás) para encontrar el cliente más antiguo con antigüedad = k en $O(\log n)$. Este enfoque es superior a búsqueda lineal $O(n)$ porque divide el espacio de búsqueda recursivamente a la mitad. La garantía asintótica $O(\log n)$ es fundamental para consultas parametrizadas. El backtracking en MergeSort permite recuperar la solución original si es necesario.

- **Módulo B:** En este módulo resolvemos el árbol de expansión de costo mínimo (MST). El paradigma que usamos es codicioso (greedy).
Justificación: El algoritmo de Kruskal se basa en la propiedad de intercambio de MST: en cada paso, seleccionar la arista de menor costo que no forma ciclo garantiza la solución óptima. La estructura Union-Find valida que no existan ciclos, permitiendo complejidad $\Theta(m \log m)$ donde m = número de aristas. Este problema no admite programación dinámica porque la decisión de incluir una arista es independiente de decisiones previas.
- **Módulo C:** En este módulo resolvemos el problema de mochila 0-1 con capacidad limitada. El paradigma que usamos es programación dinámica.
Justificación: A diferencia del módulo B, aquí el enfoque codicioso (seleccionar por mejor ratio valor/peso) NO garantiza optimalidad. Necesitamos explorar subestructura óptima: cada celda $dp[i][w]$ contiene la solución máxima usando los primeros i elementos con capacidad w . El principio de optimalidad de Bellman asegura que la solución óptima contiene subsoluciones óptimas. La complejidad pseudopolinomial $\Theta(n \cdot W)$ es aceptable porque $W=500$ es manejable. El backtracking recupera la solución en $O(n)$.

2 Descripción del Dataset

2.1 Estadísticas generales

Cargamos 7,043 registros de clientes desde un archivo CSV. Todos los registros fueron procesados exitosamente. El dataset contiene 21 atributos por cliente, pero nos enfocamos en 5 clave.

2.2 Manejo de Valores Nulos

Durante la carga, detectamos que 11 registros (0.16%) tenían el campo TotalCharges vacío o con espacios en blanco. Esto es común en datos reales. Decisión de imputación: Convertir estos 11 valores a 0.0 (cliente sin cargos registrados históricamente). Esto es aceptable porque:

- Representa solo 0.16% de los datos (mínimo impacto)
- Es conservador: asumimos que el cliente no ha gastado
- No distorsiona análisis: un cliente sin cargos simplemente tiene peso 0 en mochila

El código que lo hace:

```
if (value.empty() || value == " ") {  
    nullCounter++;  
    return 0.0;  
}
```

2.3 Construcción del Grafo para Módulo B

Para el Módulo B necesitamos construir un grafo de 20 nodos donde cada nodo representa una zona geográfica. Aquí cómo lo hacemos:

Formación de nodos: Distribuimos los 7,043 clientes entre 20 nodos usando round-robin. Cliente 0 -- Nodo 0, Cliente 1 -- Nodo 1, ..., Cliente 20 -- Nodo 0, etc. Esto da ~352 clientes por nodo y es determinístico (mismos resultados siempre).

Peso de cada nodo: Cada nodo k obtiene un peso que representa el promedio de los cargos mensuales de sus clientes. Este peso es lo que usamos para calcular distancias entre nodos. Los pesos son valores USD reales.

Aristas y sus pesos: Creamos un grafo completo K_{20} , es decir, cada nodo está conectado con todos los demás (190 aristas totales). El peso de cada arista (u,v) representa la "distancia" o "costo" entre dos zonas.

3 Módulo A – Divide y vencerás

3.1 ¿Por qué divide y vencerás para este problema?

Tenemos dos tareas:

1. Ordenar los 7,043 registros por antigüedad ascendente.
2. Responder 5 consultas binarias: ¿Quién es el cliente más antiguo con antigüedad = k ?

Si ordenamos con MergeSort ($O(n \log n) = 95,000$ operaciones) y luego buscamos con búsqueda binaria ($O(\log n) = 13$ comparaciones por consulta), gastamos muy poco tiempo total.

Si usáramos búsqueda lineal sin ordenar, sería $O(n)=7,043$ comparaciones por cada consulta lo que sería igual a 35,215 comparaciones solo para las 5 consultas. Esto es 3.7 veces más lento.

El principio de divide y vencerás: Divide el problema en mitades, lo resuelve mediante la recursividad, y luego combina las soluciones. Esto funciona porque:

- **MergeSort:** Divide mitades, ordena recursivamente cada mitad, y al final junta las soluciones.
- **Búsqueda binaria:** Divide el espacio de búsqueda a la mitad en cada iteración.

3.2 MergeSort

MergeSort es un algoritmo de divide y conquista que lo que hace es dividir recursivamente el arreglo en mitades hasta obtener subarreglos de un elemento, luego fusiona en orden ascendente.

Complejidad: $O(n \log n)$ en el mejor, promedio y peor caso.

Características del resultado:

- Ordena de forma ascendente por antigüedad.
- Primer registro: antigüedad 0.
- Último registro: antigüedad 72.
- Mantiene estabilidad relativa, es decir, siempre preserva el orden relativo de los elementos con valores o claves iguales entre la entrada y salida.

3.3 Búsqueda binaria

La búsqueda binaria es un algoritmo que busca un valor exactamente igual a la antigüedad consultada.

Especificación:

- **Busca:** $\text{antigüedad} == k$ (igualdad exacta).
- **Retorna:** índice del registro encontrado.
- **Si no encuentra el registro:** retorna -1.

Complejidad: $O(\log n)$

3.4 Resultados de búsqueda (Archivo búsquedas_A.txt)

```
Q_A01 k=72 idCliente=0784-ZQJZX antigüedad=72
Q_A02 k=60 idCliente=7581-EBBOU antigüedad=60
Q_A03 k=45 idCliente=6568-POCUI antigüedad=45
Q_A04 k=30 idCliente=1421-HCERK antigüedad=30
Q_A05 k=12 idCliente=6789-HJBWG antigüedad=12
```

4 Modulo B: Algoritmo Greedy (Kruskal MST)

4.1 ¿Por qué Kruskal (Greedy) para este problema?

Kruskal selecciona iterativamente la arista de menor peso que no forma ningún ciclo. Esto tiene un poco de miopía porque solamente mira localmente, pero es matemáticamente óptimo.

4.2 Problema del árbol generador mínimo

Construye un árbol generador mínimo de un grafo ponderado:

- **Nodos:** 20 clientes.
- **Aristas:** 190 generadas (distancias Euclidianas).

4.3 Algoritmo de Kruskal

Pasos:

- Generar todas las aristas: $n(n-1) / 2 = 190$
- Ordenar por peso ascendente
- Usar Union-Find para detectar ciclos
- Agregar aristas, pero solamente si conecta componentes distintos (Es decir no genera ciclos)

Complejidad: $O(m \log m)$, donde m son las 190 aristas.

4.4 Resultados (Archivo mst_red.txt)

- Nodos: 20
- Aristas en MST: 19
- Peso total: 2,388
- Costo promedio: 129,037

El MST representa la red óptima con el mínimo costo total.

5 Modulo C: Programación dinámica (Knapsack)

5.1 ¿Por qué programación dinámica?

En este problema la programación dinámica es obligatoria, esto debido a que:

- Greedy falla, porque toma la mejor decisión local en cada paso, pero en los problemas NP, eso casi nunca nos va a garantizar la mejor solución global.
- Fuerza bruta es intratable, esto debido a que el espacio de búsqueda crece de forma exponencial. Para un tamaño de $n=50$, el número de combinaciones (2^{50}) supera la magnitud de la capacidad de procesamiento en un tiempo razonable, lo que hace la dificultad de los problemas NP.
- La subestructura óptima existe por lo que es posible aplicar programación dinámica

5.2 Problema de asignación del ancho de banda

El contexto es que la empresa de telecomunicaciones debe de asignar ancho de banda limitado (exactamente 500 unidades) entre 50 clientes activos. Cada cliente tiene un peso (`cargosTotales`) y un valor (proporcional a su contribución económica y lealtad).

Restricción fundamental: Este es un problema NP-Completo. No hay ningún algoritmo que resuelva este problema en tiempo polinomial. Por lo que la programación dinámica es la mejor opción.

5.3 Cálculo del valor y del peso

Peso: Se define como los “`cargosTotales`” del cliente.

$$peso = cargosTotales$$

Valor: Se define como una función de los “`cargosTotales`” ponderado por un factor de lealtad basado en antigüedad.

$$valor = cargosTotales \times \left(1 + \frac{antigüedad}{100}\right)$$

Obtención de clientes activos:

- **Filtro:** Clientes que no se van (rotación = No)
- **Total extraídos:** 50 clientes de los 7,043 originales del dataset

5.4 Algoritmo DP Knapsack

En este algoritmo en lugar de probar todas las combinaciones como lo haría fuerza bruta, lo que hacemos es construir la tabla DP que lo que hace es que construye una tabla la cual reutiliza subproblemas resueltos anteriormente.

Tabla DP:

dp [i][w] = La mejor ganancia si considero los primeros “i” clientes y si tengo exactamente “w” unidades de espacio disponible.

La complejidad temporal y espacial coinciden en $O(n \times W)$ porque el algoritmo construye una matriz de $(n+1) \times (W+1)$ celdas donde cada una representa un subproblema resuelto con una operación simple. En nuestro caso, $51 \times 501 = 25,551$ operaciones (~25 microsegundos). Este crecimiento pseudopolinomial es

altamente eficiente comparado con alternativas exponenciales, permitiendo resolver problemas NP-difíciles en tiempo práctico.

Comparación con fuerza bruta

Enfoque	Complejidad	Operaciones	Tiempo
Fuerza bruta	$O(2^n)$	10^{15}	317 años
DP Knapsack	$O(n \times W)$	25,551	0,025 ms

5.5 Resultados de Knapsack (Archivo asignación_bw.txt)

- **Valor total maximizado:** 9,611 unidades
- **Número de clientes seleccionados:** 19
- **Ancho de banda utilizado:** 500 / 500 (Usamos el 100 %)

6 Integración del pipeline

Flujo:

- Leer CSV – 7,043 solicitudes
- MergeSort (ascendente por antigüedad)
- Guardar solicitudes_ordenadas.csv
- Búsqueda binaria
- Kruskal MST (20 primeros nodos) – mst_red.txt
- Extraer 50 clientes activos (Es decir rotación= No)
- DP Knapsack – asignación_bw.txt

7 Conclusiones

- Durante el desarrollo de este proyecto pudimos comprender de una forma más práctica la importancia de aplicar diferentes paradigmas algorítmicos. Más allá de la teoría, fue posible observar cómo conceptos como la complejidad Big-O realmente se reflejan en la ejecución del código y en el comportamiento de cada algoritmo. También aprendimos que no basta con implementar bien un algoritmo, sino que es necesario saber elegir el paradigma adecuado según el tipo de problema. Por ejemplo, un algoritmo de ordenamiento como MergeSort puede ser muy eficiente para organizar datos, pero no sería útil para resolver un problema de optimización como Knapsack. De la misma manera, la programación dinámica no tendría sentido para resolver un árbol de expansión mínima, donde greedy es más apropiado.

- En general, este trabajo fortaleció nuestra comprensión sobre la importancia de los paradigmas algorítmicos en la ingeniería. También nos queda super claro que la correcta aplicación de un algoritmo no está solo en programar, sino que debemos de saber analizar el problema, de seleccionar el enfoque adecuado y de validar que los datos obtenidos sean los correctos. Por eso, pensamos que los algoritmos siguen y seguirán siendo una base fundamental para el desarrollo de sistemas escalables y eficientes.

8 Herramientas

- Lenguaje: C++
- Compilador: g++
- **Asistencia de la inteligencia Artificial:** Se utilizó Claude (modelo Haiku 4.5) de Anthropic como herramienta complementaria en tres aspectos:
Aclaración de Conceptos: Explicaciones paso a paso de Programación Dinámica y Union-Find mediante ejemplos concretos que permitieron visualizar cómo funcionaba la tabla $DP[i][w]$ y la detección de ciclos, clarificando la lógica antes de implementar.
Mejora de Redacción: Asistencia en reestructurar secciones complejas del informe (especialmente Módulo C) para mantener precisión técnica.
Implementación de Funciones: Apoyo en el desarrollo de dos funciones críticas: backtracking en Knapsack (reconstruir solución a partir de tabla DP) y corrección de búsqueda binaria (cambio de $\geq$ a $==$ con ternary branching correcto), proporcionando pseudocódigo anotado que facilitó la traducción correcta a C++.

9 Referencias

- Cormen, T. H., et al. (2009). Introduction to Algorithms (3rd ed.). MIT Press.

- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.
- Erickson, J. (2019). *Algorithms*. University of Illinois. Retrieved
- Kaggle Datasets. Telecom Customer Churn.
<https://www.kaggle.com/datasets/blastchar/telco-customer-churn>

10 Roles del equipo

Rol	Juan Manuel Gallo	David García
Diseño de los algoritmos	Responsable	Responsable
Implementación del módulo A	Responsable	
Implementación del módulo B		Responsable
Implementación del módulo C	Responsable	Responsable
Documentación técnica	Responsable	Responsable